# Insights Summary

The most important takeaways from the claude code telemetry analysis.

## 1. Platform-Level Activity and Cost

### Overview

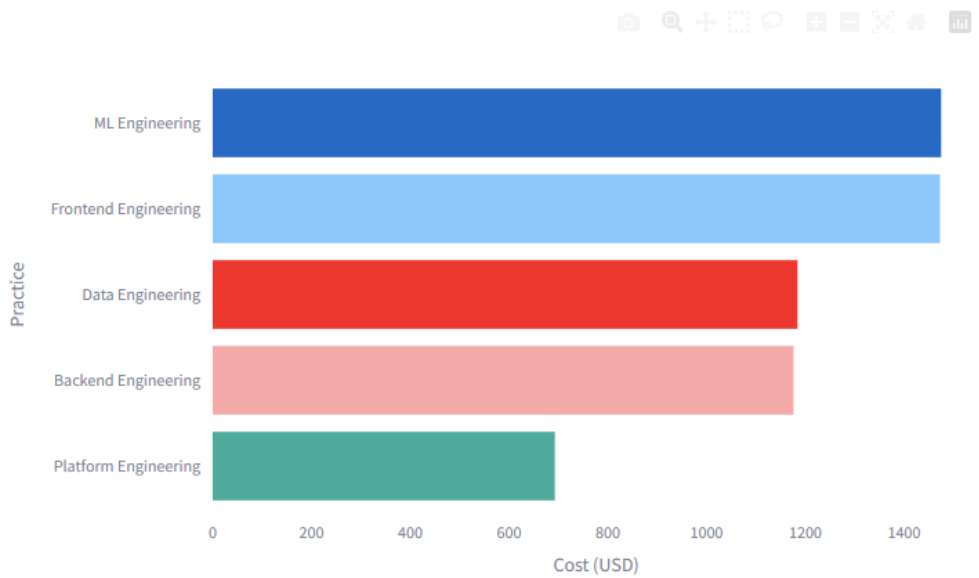
| Total Events | Sessions | Active Users | Total Cost |
|---|---|---|---|
| 454,428 | 5,000 | 100 | $6,001.43 |

- Daily cost is relatively stable around $100/day (avg $100.02, min $64.66, max $161.41), **indicating consistent system usage rather than sporadic spikes.**
- Daily active users are also stable (avg 56.9, range 45-72), **suggesting healthy recurring engagement**.

## 2. Cost Drivers

- Top cost-driving practices are **ML Engineering** and **Frontend Engineering** (nearly tied), followed by Data Engineering.

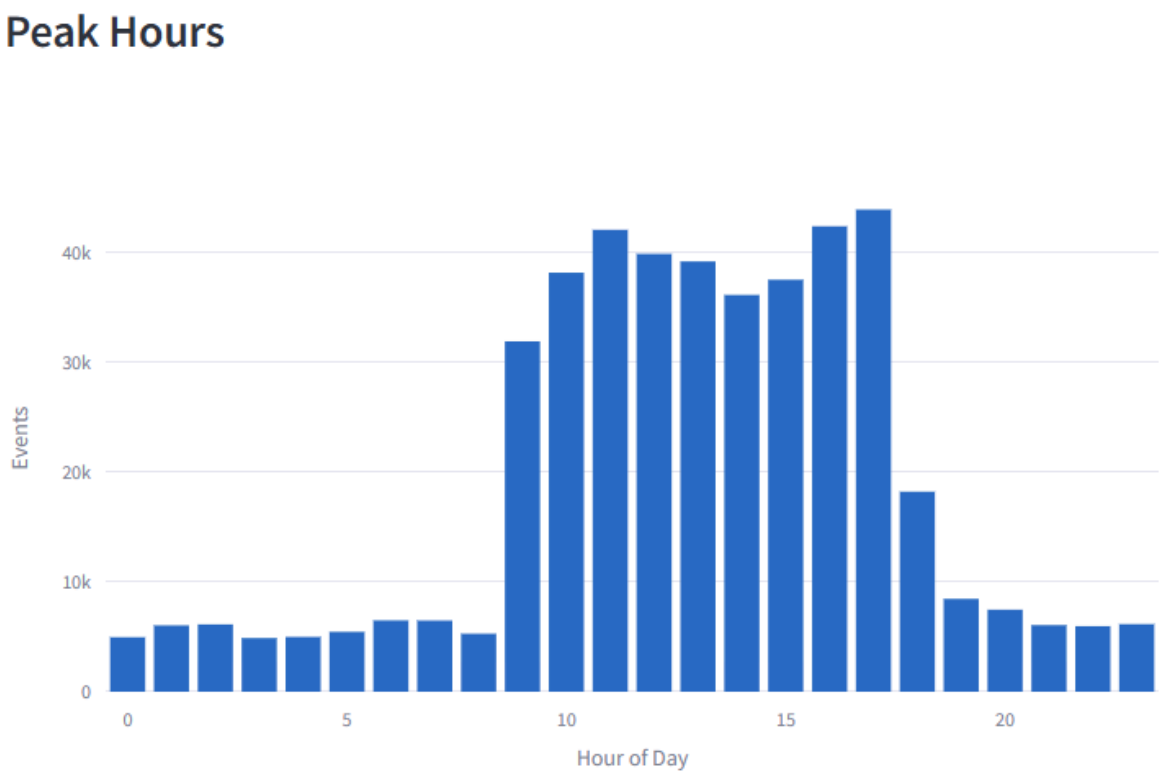
### Cost by Practice



- At user level, top spenders are tightly clustered (roughly low-$90 totals each over the window), **indicating no single user dominates spend disproportionately**.

## 3. Usage Behavior and Work Rhythm

- **The highest activity concentration occurs at 17:00** (local telemetry hour aggregation), indicating a clear late-afternoon usage peak.




- Session intensity is high for heavy users: among top sessions, average size is ~705 events/session (median 642.5, max 1344), **consistent with iterative/agentic workflows rather than short one-shot interactions**

## 4. Tooling Effectiveness and Friction Points

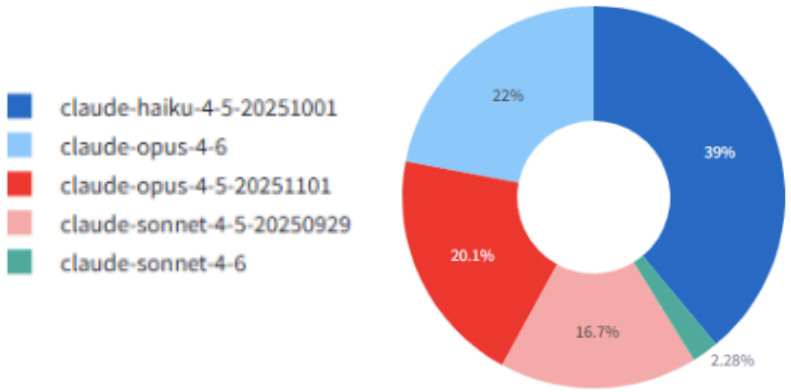
- Most-used tools: **Read, Bash, Edit, Grep, Glob, mcp_tool**.
- Lowest success rates are observed in Bash, WebSearch, and WebFetch relative to other tools.
- **This suggests environment/external-system operations are the primary reliability bottleneck vs internal file/navigation operations**.

## 5. Model Mix: Volume vs Spend

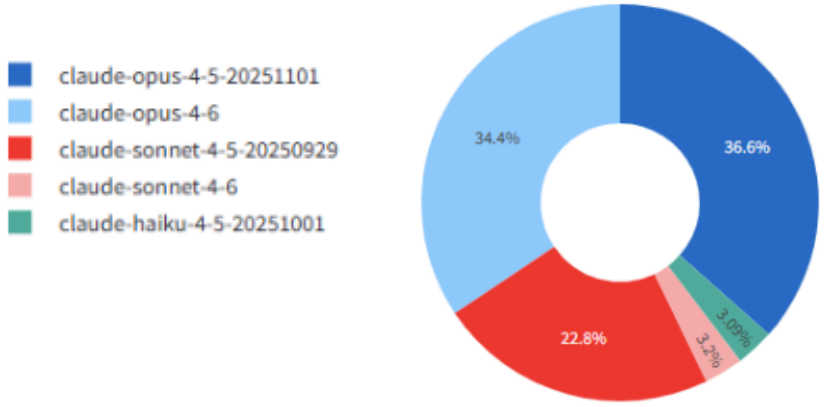
- By request count, **claude-haiku-4-5** is most used.

### Model Popularity

**By Request Count**




**By Total Cost**



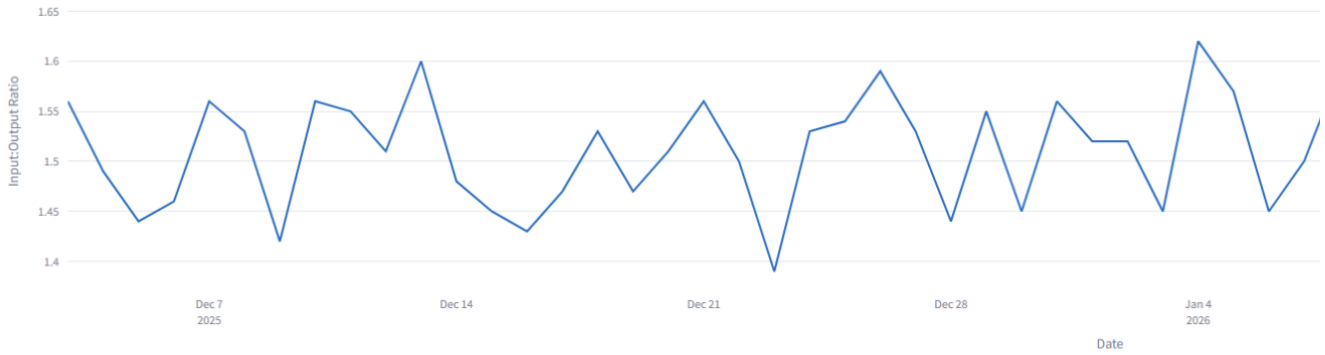

- By total cost, **claude-opus** variants dominate.

- Interpretation: **cheaper/faster models are preferred for volume, while premium models capture a larger share of spend, likely for complex tasks.**

## 6. Token Economics and Efficiency

- Average daily token flow is large and consistent:
- Input/output ratio averages 1.52 (range 1.39-1.67), **showing input generally exceeds generated output.**
- To clarify, apart from the user prompt,  the input tokens in this case include also **text conversation/history context, system/instruction text, tool/result context included in the request payload**

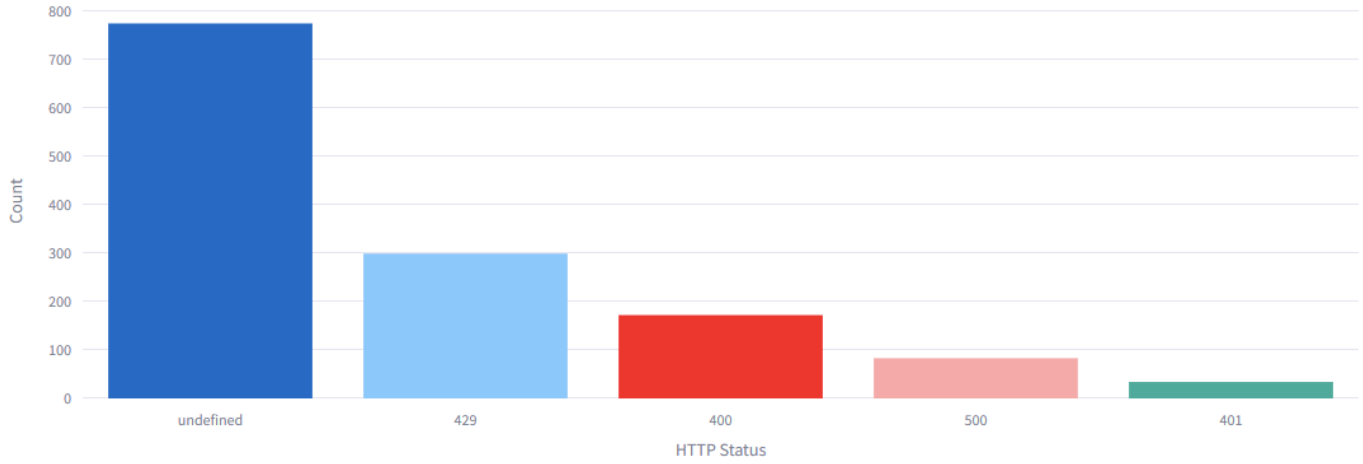
**Input / Output Token Ratio**




## 7. Error Patterns and Recovery

- Error rate averages 22.7/day (max 36/day), indicating manageable but persistent failure volume.
- Error statuses are led by undefined and 429 (rate-limit) classes, followed by 400.

**Errors by Status Code**

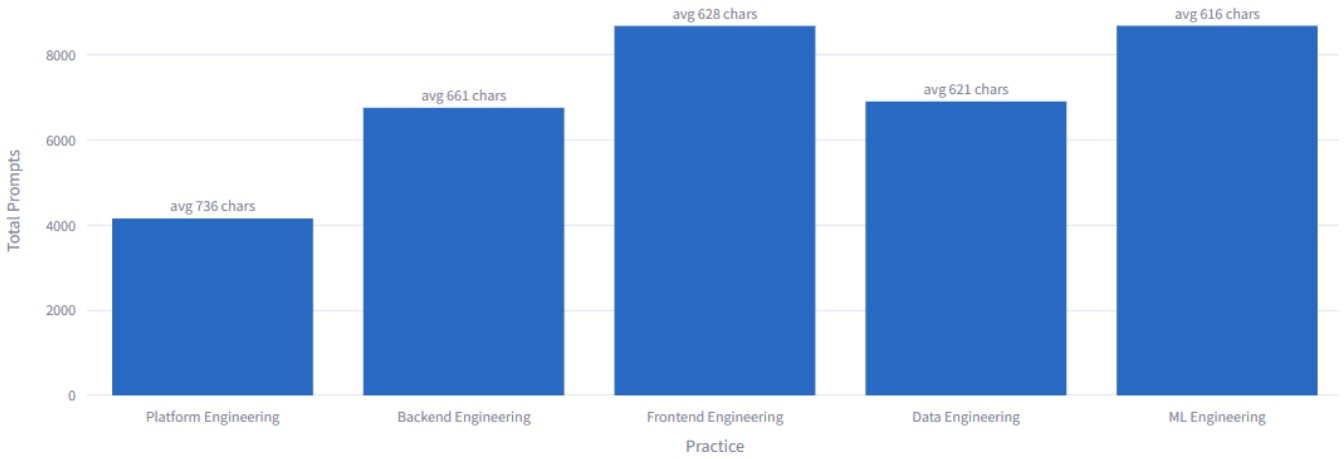


- Retry distribution is front-loaded at attempt 1, with smaller tails at attempts 2 and 3, **indicating most failures are either resolved quickly or not repeatedly retried.**
- Top error message is “Request was aborted.”, then rate-limit-related messaging, **pointing to cancellation/interrupt and throughput limits as dominant failure modes**.

## 8. User Behavior and Prompt Distribution

- Prompt length is highly skewed: **Mean: 642.7, Median: 127.0, Max: 186,680**
- **The large mean-median gap indicates a long-tail distribution with rare very large prompts.**
- Platform Engineering shows the highest average prompt length among practices, possibly reflecting architecture-heavy or context-rich workflows.

### Prompts by Practice




## Practical Takeaways

System usage is sustained and structured, not bursty. The cost is primarily driven by practice/model mix, not isolated user outliers. Caching is performing well and should remain a strategic optimization lever.

Reliability improvements should prioritize:

- shell/external-tool execution robustness
- rate-limit mitigation strategies
- clearer handling of abort/cancel flows

Current predictive module is credible as a baseline and provides meaningful forward-looking signal for planning.